

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ГЕОДЕЗИИ И
КАРТОГРАФИИ»
(ММИГАиК)
Факультет геоинформатики и информационной безопасности
(ФГиИБ)

ПРАКТИЧЕСКАЯ РАБОТА №1

**по дисциплине: Методы машинного обучения в задачах обработки
пространственных данных**

Работу выполнил:
Студент 2025-ФГиИБ-ПИ-1м
Сабитов Р.Р.
Преподаватель: Шириникин С.Г.

Москва, 2026 г.

Содержание

Введение.....	3
Практическая часть	4
Анализ длин ответов	4
Анализ содержимого при смене режима.....	5
Влияние конкретных параметров	6
Различия моделей при одинаковых режимах и промтах	6
Заключение	8
Список использованных источников	9
Приложения	10

Введение

В рамках настоящей работы был выполнен анализ результатов эксперимента по сравнению двух языковых моделей в нескольких режимах генерации: `strict`, `balanced` и `creative`. Исследование было направлено на выявление того, как параметры генерации, включая `temperature`, `top_p`, `repeat_penalty` и `num_ctx`, влияют на длину, структуру и содержательное качество ответов. Особое внимание уделялось различиям в поведении моделей при выполнении типовых подзадач: `explain`, `bugfix`, `json` и `story`.

Актуальность темы определяется тем, что современные большие языковые модели все чаще используются в учебной, исследовательской и прикладной деятельности, где важны не только корректность ответа, но и его стиль, вариативность и устойчивость к изменению параметров генерации. Анализ таких характеристик позволяет лучше понимать принципы работы моделей и выбирать режимы, соответствующие конкретной задаче.

Практическая часть

Модель	Режим	Промпт	Длина ответа
gemma3:1b	strict	explain	2726
gemma3:1b	balanced	explain	2229
gemma3:1b	creative	explain	2545
qwen3:4b	strict	explain	1200
qwen3:4b	balanced	explain	1295
qwen3:4b	creative	explain	1206

Таблица 1 – Сравнение длин ответов

Анализ длин ответов

В сводной таблице хорошо видно, что при переходе от strict → balanced → creative длина ответов у обеих моделей растёт, особенно для промптов explain и bugfix. Это ожидаемо: более высокая temperature и, у gemma, снижение repeat_penalty ослабляют «сжатость» и делают модель более многословной и вариативной. Для json длина почти не меняется между режимами, потому что формат сильно зафиксирован промптом, и пространство вариантов маленькое.

Если смотреть на модели, qwen3:4b в среднем даёт чуть более компактные explain, но заметно более длинные bugfix (особенно при strict/balanced), где она добавляет дополнительные пояснения и варианты обработки пустого списка. В творческих story qwen часто выдаёт немного более длинный текст, чем gemma, особенно в creative, что согласуется с более крупной моделью и большим num_ctx в двух режимах.

Из этого видно, что у gemma связь «creative → всегда длиннее» не монотонная: strict уже переходит в лёгкое словесное избыточество, а в balanced/creative длины пляшут вокруг одной планки. У qwen различия между режимами по explain минимальны, и она остаётся довольно компактной даже в creative — режим влияет больше на стиль, чем на длину.

Анализ содержимого при смене режима

В `explain` для бинарного поиска `strict`-ответы у обеих моделей ближе всего к учебниковому описанию, но при этом у `gemma` в `strict` заметно больше неточностей в интуитивной части и формальных шагах (например, перепутанные условия цикла и диапазоны поиска). `balanced` у `gemma` делает структуру более аккуратной, добавляет важное условие «список должен быть отсортирован» и в целом улучшает согласованность текста; `creative`, наоборот, добавляет странные формулировки и логические ошибки (условия `while` инвариансируются, описание шагов путается).

У `qwen` структура объяснения бинарного поиска во всех трёх режимах практически совпадает по шагам алгоритма; различия в том, насколько разговорная и «продвинутая» интуитивная часть (логарифмическая сложность, акцент на отсортированности и т. п.). То есть для `qwen temperature` меняет стиль, но не ломает алгоритмическую корректность, а для `gemma` рост `temperature` заметно увеличивает риск логических сбоев даже при похожей длине.

В `bugfix`-подзадаче обе модели во всех режимах корректно ловят главную проблему — инициализацию `max_val` через 0 и кейс с отрицательными числами/пустым списком, но паттерны разные. `gemma` в `strict/balanced/creative` почти всегда предлагает инициализацию через `lst +` опциональную проверку на пустой список, тогда как `qwen` часто идёт через `float('-inf')` и явно обсуждает поведение на пустом списке, в `strict` иногда добавляя два варианта решения.

Для `story` различия режимов наиболее «осознаемы»: `strict` у обеих моделей даёт более простые, сдержанные истории с минимальным сюжетом, `balanced` добавляет атмосферу и мета-комментарии про LLM/ программистов, а `creative` уходит в более сюрреалистичные, местами рваные по логике тексты. `gemma` в `creative` сильнее уезжает в бессвязность, `qwen` сохраняет связность, но усиленно нагнетает драму, рефлексии и «мета»-слой про LLM и разработчика.

Влияние конкретных параметров

Температура. Для gemma переход $0.1 \rightarrow 0.4 \rightarrow 0.9$ проявляется как увеличение разнообразия формулировок, но ещё сильнее — как рост нестабильности структуры: алгоритмы и формальные списки в creative могут содержать противоречащие друг другу условия и описания. У qwen даже при temperature 0.9 структура алгоритма бинарного поиска остаётся инвариантной, а вариативность уходит в «обёртку» — интуитивное объяснение, примеры, словарь.

top_p. В обоих случаях стоит довольно высоким (0.9–0.95), поэтому его влияние маскируется температурой и промптом; ты не тестировал жёсткие срезы вроде 0.5, и по этим данным нельзя отделить вклад top_p от temperature. repeat_penalty присутствует только у strict у обеих моделей (1.1) и, судя по текстам, слегка уменьшает вербализм, но не кардинально: и при 0.1/1.1 gemma всё равно генерирует избыточные объяснения.

num_ctx. В balanced у обеих моделей num_ctx вдвое больше (8192), однако длины ответов в balanced не вдвое, и даже не монотонно больше strict/creative. Это подтверждает, что num_ctx в твоём эксперименте не является лимитирующим фактором: ответы намного короче контекстного окна, и отличие num_ctx лишь даёт запас, но не меняет поведение при таких промптах.

Различия моделей при одинаковых режимах и промптах

При одинаковых параметрах strict qwen показывает более высокую алгоритмическую дисциплину и более явно «инженерный» стиль: чёткие шаги, корректная инициализация, обсуждение крайних кейсов, аккуратные сигнатуры функций. gemma, наоборот, даёт больше «объяснительного» текста, но легче допускает концептуальные неточности, особенно когда промпт просит и интуитивную, и формальную часть.

В json-промпте обе модели выдают почти идентичные структуры JSON, но различаются в числовых значениях total: у gemma в strict-balanced-creative всегда 25 ($3 \times 10 + 2 \times 15$), у qwen — 60 (по сути, ошибка, если считать как сумму). Это хороший пример того, что более крупная модель (qwen3:4b) не

обязательно точнее в арифметике, особенно при повышенной температуре, и что структурный формат не гарантирует корректность содержимого.

В story-промпте обе модели явно подхватывают стилистические паттерны (ночь, код, дождь, LLM, программист), но qwen устойчиво строит более цельный нарратив; у gemma в creative стилистика иногда скатывается в бессвязный поток образов. При этом в strict/balanced gemma выдаёт довольно читабельные, но более шаблонные истории, а qwen добавляет больше «рефлексии» и мета-уровня.

Заключение

Проведенный анализ показал, что параметры генерации существенно влияют на характер ответов языковых моделей, однако степень этого влияния различается в зависимости от конкретной модели и типа задания. Для объяснительных и творческих задач изменение режима strong/balanced/creative заметно отражается на длине, стиле и устойчивости логики ответа, тогда как в форматированных задачах, таких как json, эффект параметров выражен слабее.

Исследование также подтвердило, что более многословный ответ не всегда означает более качественный. Одна модель демонстрировала большую подробность, но чаще допускала смысловые неточности, тогда как другая была более строгой в алгоритмическом отношении и лучше сохраняла структуру решения. Следовательно, выбор модели и режима генерации должен определяться задачей: для учебного объяснения, программной отладки или творческого текста оптимальными будут разные настройки.

Таким образом, исходный эксперимент позволяет сделать вывод о том, что режимы генерации и параметры sampling оказывают значимое, но неодинаковое влияние на поведение языковых моделей. Полученные результаты могут быть использованы при выборе конфигурации модели для практических задач, где важны либо точность и структурность, либо выразительность и вариативность текста.

Список использованных источников

1. MachineLearningPlus. LLM Temperature, Top-P, and Top-K Explained [Электронный ресурс]. – 2024. – URL: <https://machinelearningplus.com/gen-ai/llm-temperature-top-p-top-k-explained/> (дата обращения: 06.06.2026).
2. Prompting Guide. LLM Settings [Электронный ресурс]. – 2024. – URL: <https://www.promptingguide.ai/introduction/settings> (дата обращения: 06.06.2026).
3. OpenAI. API Reference: Chat Completions [Электронный ресурс]. – 2024. – URL: <https://platform.openai.com/docs/api-reference/chat> (дата обращения: 06.06.2026).
4. Hugging Face. Generation Strategies for Language Models [Электронный ресурс]. – 2024. – URL: <https://huggingface.co/blog/how-to-generate> (дата обращения: 06.06.2026).
5. Gemma-3 Release [Электронный ресурс]. – 2025. – URL: <https://huggingface.co/collections/google/gemma-3-release> (дата обращения: 06.06.2026).
6. Qwen Team. Qwen Models and Chat Platform [Электронный ресурс]. – 2025. – URL: <https://qwenlm.github.io/> (дата обращения: 06.06.2026).

Приложения

Приложение 1 – Вывод скрипта для анализа ответов нейросетевых моделей

```
# Ollama Эксперимент: 20260313_092159
```

```
## Конфигурация
```

```
- Модели: gemma3:1b, qwen3:4b
```

```
- Промпты: 4 шт.
```

```
- Наборы параметров: ['strict', 'balanced', 'creative']
```

```
## Модель: `gemma3:1b`
```

```
### Режим: strict | Параметры: {'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9, 'repeat_penalty': 1.1}
```

```
#### explain
```

```
**Параметры:** `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9, 'repeat_penalty': 1.1}`
```

```
...
```

```
## 1. Интуитивное объяснение для новичка
```

Представь, что у тебя есть список чисел и ты хочешь найти конкретное число в этом списке. Бинарный поиск - это как искать в списке, только вместо того, чтобы проверять каждое число по очереди, мы проверяем, является ли текущее число больше или меньше предыдущего. Мы делаем это, двигаясь "вниз" или "вверх" по списку, пока не найдем нужный элемент. Это очень эффективный способ поиска, особенно когда список большой.

```
## 2. Формальное описание шагов в виде списка
```

1. ****Инициализация:**** Начинаем с начального значения (например, первого элемента списка).
2. ****Сравнение:**** Сравниваем текущее число с целевым числом.
3. ****Уменьшение диапазона:**** Если текущее число меньше целевого, уменьшаем диапазон поиска (на одну позицию вниз).
4. ****Повторение:**** Повторяем шаги 2 и 3 до тех пор, пока не найдем целевое число или достигнем конца списка.
5. ****Возврат:**** Возвращаем найденное число.

```
## 3. Код на Python
```

```
```python
```

```
def binary_search(list_to_search, target):
```

```
 """
```

```
 Выполняет бинарный поиск в списке.
```

```
 Args:
```

```
 list_to_search: Список чисел для поиска.
```

```
 target: Число, которое нужно найти.
```

```
 Returns:
```

```
 Индекс найденного числа или -1, если число не найдено.
```

```

"""
low = 0
high = len(list_to_search) - 1

while low <= high:
 mid = (low + high) // 2 # Вычисляем середину диапазона
 guess = list_to_search[mid]

 if guess == target:
 return mid # Нашли! Возвращаем индекс
 elif guess > target:
 high = mid - 1 # Ищем в левой половине
 else:
 low = mid + 1 # Ищем в правой половине

return -1 # Число не найдено
"""

```

**\*\*Как работает код:\*\***

```

* `binary_search(list_to_search, target)`: Функция принимает список чисел
(`list_to_search`) и целевое число (`target`).
* `low = 0`: Устанавливаем начальный индекс поиска в начало списка.
* `high = len(list_to_search) - 1`: Устанавливаем конечный индекс поиска в
конец списка.
* `while low <= high`: Цикл продолжается, пока `low` меньше или равно
`high`. Это означает, что есть еще элементы для рассмотрения.
* `mid = (low + high) // 2`: Вычисляем середину диапазона поиска. `//` -
это целочисленное деление.
* `guess = list_to_search[mid]`: Запоминаем значение текущего элемента в
середине списка.
* `if guess == target`: Если `guess` равен целевому числу, функция
возвращает индекс `mid`.
* `elif guess > target`: Если `guess` больше целевого числа, значит,
целевое число находится в левой половине списка. Устанавливаем `high = mid -
1`.
* `else`: Если `guess` меньше целевого числа, значит, целевое число
находится в правой половине списка. Устанавливаем `low = mid + 1`.
* `return -1`: Если цикл завершается без нахождения целевого числа, функция
возвращает -1.
"""

```

#### bugfix

```

Параметры: `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9,
'repeat_penalty': 1.1}`

```

...

Вот ответы на ваши вопросы:

**\*\*1) Что делает код?\*\***

Код определяет максимальное значение в списке чисел. Он инициализирует переменную `max\_val` равной 0. Затем он перебирает все элементы списка `lst`. Для каждого элемента он сравнивает его с текущим значением `max\_val`. Если элемент больше `max\_val`, то `max\_val` обновляется до значения этого

элемента. После завершения цикла, код возвращает значение `max\_val`, которое будет содержать максимальное значение из списка.

**\*\*2) Найди ошибку.\*\***

Ошибка в том, что код не обрабатывает случай, когда все элементы списка равны. Если все числа в списке равны друг другу, то `max\_val` останется всегда равной 0, и функция вернет 0, даже если в списке есть хотя бы одно число больше нуля.

**\*\*3) Исправь.\*\***

Вот исправленный код:

```
```python
def find_max(lst):
    if not lst: # Обработка пустого списка
        return None # Или можно вернуть какое-то другое значение по
# умолчанию, например 0
    max_val = lst[0] # Инициализируем max_val первым элементом списка
    for x in lst:
        if x > max_val:
            max_val = x
    return max_val
```
```

**\*\*Объяснение изменений:\*\***

\* **\*\*`if not lst:`\*\***: Добавлена проверка, что список не пуст. Если список пуст, то функция возвращает `None`. Это предотвращает ошибку `IndexError`, которая возникла бы при попытке получить доступ к элементу пустого списка.

\* **\*\*`max\_val = lst[0]`\*\***: Инициализируем `max\_val` первым элементом списка. Это гарантирует, что даже если список содержит только один элемент, он будет правильно обработан.

Теперь код корректно обрабатывает случай, когда список пуст, возвращая `None`.

```
json
Параметры: `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9,
'repeat_penalty': 1.1}`
...
```json
{
  "buyer": "Петя",
  "items": [
    {"name": "яблоко", "count": 3, "price": 10},
    {"name": "апельсин", "count": 2, "price": 15}
  ],
  "total": 25
}
...
```
```

```
story
Параметры: `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9,
'repeat_penalty': 1.1}`
```

```
...
```

```
Ночь кода
```

Холодный дождь хлестал по окну, отражая мерцающий свет монитора. Алекс, с лицом, изрезанным сетью мелких царапин и усталостью, сидел в полумраке своей комнаты. Он – программист, но не тот, кто пишет код для пользователей. Он пишет для LLM – больших языковых моделях, которые он, по сути, разводит на ночь.

Он ковырялся в коде, словно пытался вытащить из него душу. Сегодня был очередной "продвинутый" запрос: написать короткую историю о потерянном коте. Алекс вздохнул. Все эти запросы – как старые, заброшенные проекты. Он уже давно перестал понимать, зачем вообще это делает.

Он запустил скрипт, и на экране появилось текст: "Котик по имени Скрэтч, он пропал в библиотеке. Скрэтч был очень любопытным." Алекс усмехнулся. Он уже давно перестал думать о том, что это \*действительно\* программа. Просто набор правил и вероятностей. Он просто хотел, чтобы модель выдала что-то... хоть что-нибудь.

Затем, на экране появилось сообщение: "Пожалуйста, добавьте больше деталей о библиотеке." Алекс замер. Он знал, что это – лишь очередная попытка. Но в этой бесконечной череде случайных слов, он вдруг почувствовал странное, почти... удовлетворение. Может быть, это и есть его работа.

```
...
```

```

```

```
Режим: balanced | Параметры: {'temperature': 0.4, 'num_ctx': 8192,
'top_p': 0.9}
```

```
explain
```

```
Параметры: `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`
```

```
...
```

```
1. Интуитивное объяснение для новичка (до 5 предложений)
```

Бинарный поиск - это способ найти элемент в отсортированном списке (или массиве) по определенному критерию. Он работает, как будто мы ищем в книге, начиная с середины. Если мы дошли до середины, и наткнулись на элемент, то мы нашли его. Если мы дошли до конца, то элемент не найден, и мы начинаем искать в следующем месте. Ключевая идея - поиск в середине, что значительно быстрее, чем перебирать весь список.

```
2. Формальное описание шагов в виде списка
```

1. **Подготовка:** Убедитесь, что список отсортирован по указанному критерию.
2. **Определение начальной позиции:** Выберите начальную позицию, в которой мы будем искать.
3. **Сравнение:** Сравните значение элемента, который мы ищем, с значением элемента в текущей позиции.

4. **\*\*Уменьшение диапазона:\*\*** Если значение элемента, который мы ищем, больше, чем значение элемента в текущей позиции, значит, элемент не находится в списке. Уменьшите начальную позицию на 1.
5. **\*\*Повторение:\*\*** Повторяйте шаги 3 и 4, пока не найдете элемент или не достигнете максимальной глубины поиска (которая ограничена).
6. **\*\*Возврат:\*\*** Если элемент найден, верните его. Если элемент не найден, верните -1 (или какое-то другое значение, указывающее на отсутствие элемента).

## 3. Код на Python

```
```python
def binary_search(list_to_search, target):
    """
    Ищет элемент в отсортированном списке методом бинарного поиска.

    Args:
        list_to_search: Отсортированный список, в котором нужно искать.
        target: Элемент, который нужно найти.

    Returns:
        Индекс элемента, если он найден, иначе -1.
    """
    low = 0
    high = len(list_to_search) - 1

    while low <= high:
        mid = (low + high) // 2 # Вычисляем середину
        if list_to_search[mid] == target:
            return mid # Элемент найден, возвращаем индекс
        elif list_to_search[mid] < target:
            low = mid + 1 # Ищем в правой половине
        else:
            high = mid - 1 # Ищем в левой половине

    return -1 # Элемент не найден
```
```

**\*\*Как использовать:\*\***

```
```python
my_list = [2, 5, 7, 8, 11, 12]
target_value = 11
result = binary_search(my_list, target_value)

if result != -1:
    print("Элемент найден по индексу:", result)
else:
    print("Элемент не найден.")
```

bugfix
Параметры: `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`
```
```

Вот ответы на ваши вопросы:

****1) Что делает код?****

Код определяет максимальное значение в списке чисел. Он инициализирует переменную `max_val` значением 0. Затем он перебирает все элементы в списке `lst`. Для каждого элемента он сравнивает его с текущим значением `max_val`. Если текущий элемент больше `max_val`, то `max_val` обновляется до текущего значения элемента. После завершения цикла, код возвращает значение `max_val`, которое является максимальным значением в списке.

****2) Найди ошибку.****

Ошибка в том, что код не обрабатывает случай, когда все элементы в списке равны. Если все элементы равны, то `max_val` останется всегда 0, и функция вернет 0, что не является правильным результатом.

****3) Исправь.****

Вот исправленный код:

```
```python
def find_max(lst):
 if not lst: # Обработка пустого списка
 return None # Или можно вернуть какое-то другое значение, например,
0
 max_val = lst[0] # Инициализируем max_val первым элементом списка
 for x in lst:
 if x > max_val:
 max_val = x
 return max_val
```
```

****Объяснение изменений:****

```
*   **`if not lst:`**: Добавлена проверка, что список не пуст. Если список
пуст, то функция возвращает None (или можно вернуть какое-то другое
значение, например, 0). Это предотвращает ошибку IndexError, которая
возникла бы, если бы мы пытались получить доступ к элементу пустого списка.
*   **`max_val = lst[0]`**: Теперь max_val инициализируется первым
элементом списка. Это гарантирует, что max_val всегда будет содержать
значение первого элемента списка, даже если все остальные элементы равны.
```
```

#### json

**\*\*Параметры:\*\*** `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`

...

```json

```
{
  "buyer": "Петя",
  "items": [
    {
      "name": "яблоко",
      "count": 3,
      "price": 10
    }
  ]
}
```

```

    },
    {
        "name": "апельсин",
        "count": 2,
        "price": 15
    }
],
"total": 25
}
...
...

```

```

#### story
**Параметры:** `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`
...
## Ночь кода

```

Свет от монитора выхватывал из темноты измученное лицо. Алекс, программист, по-настоящему усталый, сидел в своем кабинете, словно в заброшенном баре. За окном, в мерцающем неоновом свете, проплывали обрывки старых кодов – его последнее, и, пожалуй, единственное, "успешное" творение за день.

Он вглядывался в консоль, где мерцали красные сообщения об ошибках. “Ошибка 404: Не найдено.” Алекс усмехнулся, наклонился над кодом, и почувствовал, как в этом неоновом свете заливается не только свет от монитора, но и не только усталость. Он продолжил разбираться в проблеме, пока не почувствовал в какой-то не особо интересной и не очень удобной постановке.

Внезапно, в коде появилась неожиданная строка – не ошибка, а просто запись "Я программист. И я очень устало." Алекс задержался на нем на секунду, за тем и за тем в кабинете появился один один по-настоящему одинокий программист.

```

Он выключил монитор.
...

```

```

### Режим: creative | Параметры: {'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}

```

```

#### explain
**Параметры:** `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`
...

```

```

## 1. Интуитивное объяснение для новичка (до 5 предложений)

```

Бинарный поиск - это способ найти конкретный элемент в массиве (или списке) по его значению. Он работает, как будто ты ищешь пару в коробке, и когда ты нашел нужную пару, ты сразу знаешь, что ищешь. Он не ищет *все* элементы, а только *один* элемент, который соответствует твоему поисковому значению. Это очень эффективно, когда у тебя есть только один пример нужного элемента.

```

## 2. Формальное описание шагов в виде списка

```

1. ****Инициализация:**** Установите переменной ``low`` равной начальному значению (самый маленький возможный элемент в массиве). Установите ``high`` равным значению, которое соответствует значению элемента, который нужно найти (самый большой возможный элемент).
2. ****Повторяйте, пока ``high`` меньше или равно ``low``:** Продолжайте, пока ``high`` меньше или равно ``low``.
3. ****Сравнение:**** Сравните значение ``low`` и ``high``.
4. ****Изменение значений:****
 - * Если ``low`` равно ``high``: Это означает, что элемент найден. Выведите его.
 - * Если ``low`` больше, чем ``high``: Это означает, что элемент не найден. Возвращайте ``False`` (или какое-либо значение, указывающее на это).
5. ****Остановка:**** После того, как ``high`` становится равным ``low``, выйдите из цикла.

3. Код на Python

```
```python
def binary_search(arr, target):
 """
 Выполняет бинарный поиск в массиве.

 Args:
 arr: Массив (список) элементов для поиска.
 target: Элемент, который нужно найти.

 Returns:
 True, если элемент найден, False в противном случае.
 """
 low = 0
 high = len(arr) - 1

 while low <= high:
 mid = (low + high) // 2 # Вычисляем средний индекс

 if arr[mid] == target:
 return True # Элемент найден!
 elif arr[mid] < target:
 low = mid + 1 # Ищем в правой половине
 else:
 high = mid - 1 # Ищем в левой половине

 return False # Элемент не найден
```
```

****Как работает код:****

- * Функция ``binary_search`` принимает массив ``arr`` и ``target`` как аргументы.
- * ``low`` и ``high`` инициализируются как начальные значения диапазона поиска.
- * Цикл ``while`` продолжается, пока ``high`` меньше или равно ``low``.
- * ``mid`` вычисляется как средний индекс между ``low`` и ``high``.
- * Если ``arr[mid]`` равно ``target``, функция возвращает ``True`` (элемент найден).
- * Если ``arr[mid]`` меньше ``target``, функция обновляет ``low`` до ``mid + 1`` (ищем в правой половине).

```
* Если `arr[mid]` больше `target`, функция обновляет `high` до `mid - 1`
(ищем в левой половине).
* Если цикл завершается без нахождения элемента, функция возвращает
`False`.
```
```

#### bugfix

```
Параметры: `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`
```

```
```
```

1. **Что делает код?**

Код определяет максимальное значение в списке чисел. Он инициализирует переменную `max_val` равной 0. Затем он проходит по каждому числу в списке `lst`. Для каждого числа он сравнивает его с `max_val`. Если число больше `max_val`, он обновляет `max_val` до текущего числа. После завершения цикла, он возвращает `max_val`, которое будет содержать максимальное значение из списка.

2. **Найди ошибку:**

Ошибка в том, что код не обрабатывает случай, когда список пуст. Если список пуст, то `max_val` будет изначально равно 0, и цикл никогда не будет выполняться. В результате, функция вернет 0, что не является правильным результатом.

3. **Исправь:**

Вот исправленная версия кода:

```
```python
def find_max(lst):
 if not lst: # Проверяем, пуст ли список
 return None # Или можно вернуть какое-то другое значение, например,
 -float('inf')
 max_val = lst[0] # Инициализируем max_val первым элементом списка
 for x in lst:
 if x > max_val:
 max_val = x
 return max_val
```
```

Объяснение изменений:

```
* `if not lst:`: Это проверка, является ли список lst пустым. not lst
возвращает True, если список пуст, и False в противном случае.
* `return None`: Если список пуст, функция возвращает None. Это
более подходящий способ обработки пустого списка, чем просто возвращать 0.
Можно заменить None на любое другое значение, которое укажет на отсутствие
значения.
* `max_val = lst[0]`: Если список не пуст, инициализируем max_val
первым элементом списка. Это позволяет корректно обрабатывать случай, когда
список содержит только один элемент.
```

Теперь код корректно обрабатывает все возможные случаи, включая пустой список.

```

    ...

#### json
**Параметры:** `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`

    ...
```json
{
 "buyer": "Петя",
 "items": [
 {
 "name": "яблоко",
 "count": 3,
 "price": 10
 },
 {
 "name": "апельсин",
 "count": 2,
 "price": 15
 }
],
 "total": 25
}
```

#### story
**Параметры:** `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`

    ...

## Ночь кодов

Сложно сказать, сколько времени прошло, но кажется, я просидел здесь еще не один час. В комнате царит лишь приглушенный свет от монитора, висящего над собой, и запах отчаяния - смесь кофе и отработанного кода. Я - Демон Истерикив, и моя работа - починка увядающих систем. Пока что, увядают и я.

Я просматриваю очередную ветку, в которой, кажется, напрасно потратил месяцы. Все эти попытки "улучшить" LLM - просто заставить его говорить более красиво. Я, в конце концов, научился успокоить раны, а не усиливать их.

Сегодня, в одиночестве, я попытался добавить "невинное" параграф о "подарочном росте" в передаче. Просто не понимаю, как не оставлять на коде остатки этого "роста". И, правда, я не могу подождать. Жизнь программиста - это идеальная смерть.

---

**Вывод:** Внутри, программа не почитляет его. Но я уже в начале поиска последней утраты.

---

## Модель: `qwen3:4b`

```

```
### Режим: strict | Параметры: {'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9, 'repeat_penalty': 1.1}
```

```
#### explain
```

```
**Параметры:** `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9, 'repeat_penalty': 1.1}`
```

```
...
```

```
### 1) Интуитивное объяснение для новичка (до 5 предложений)
```

Бинарный поиск работает так: если у вас есть отсортированный список, вы сначала проверяете среднее значение. Если искомое число меньше середины – ищем в левой половине, если больше – в правой. Каждый шаг уменьшает область поиска в два раза, что делает поиск очень быстрым даже для больших списков.

```
### 2) Формальное описание шагов
```

1. Определите начальные границы: `left = 0`, `right = len(arr) - 1`.
2. Пока `left <= right`:
 - a. Найдите середину: `mid = (left + right) // 2`.
 - b. Сравните `arr[mid]` с искомым значением `target`.
 - c. Если `arr[mid] == target` – верните индекс `mid`.
 - d. Если `target < arr[mid]`, уменьшите правую границу до `mid - 1`.
 - e. Если `target > arr[mid]`, увеличьте левую границу до `mid + 1`.
3. Если цикл завершился без нахождения значения – верните `-1`.

```
### 3) Код на Python в одном блоке
```

```
```python
```

```
def binary_search(arr, target):
 left, right = 0, len(arr) - 1
 while left <= right:
 mid = (left + right) // 2
 if arr[mid] == target:
 return mid
 elif target < arr[mid]:
 right = mid - 1
 else:
 left = mid + 1
 return -1
```

```
...
```

```
...
```

```
bugfix
```

```
Параметры: `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9, 'repeat_penalty': 1.1}`
```

```
...
```

```
1) **Что делает код?**
```

Код находит максимальное значение в списке `lst`, но если все элементы списка отрицательные, он вернёт `0` вместо правильного максимума. Это происходит потому, что начальное значение `max\_val` инициализируется как `0`, а при обработке отрицательных чисел оно не меняется.

```
2) **Найди ошибку**
```

Ошибка в инициализации переменной `max\_val`. Начальное значение `0` некорректно для списков, содержащих только отрицательные числа. Например, для списка `[-1, -2]` функция вернёт `0`, но правильный ответ должен быть `-1`.

3) **\*\*Исправь\*\***

Исправленный код использует `float('-inf')` как начальное значение для корректной работы с отрицательными числами:

```
```python
def find_max(lst):
    max_val = float('-inf')
    for x in lst:
        if x > max_val:
            max_val = x
    return max_val
```
```

**\*\*Пояснение\*\*:**

- `float('-inf')` – это минимальное возможное значение в Python, которое гарантирует корректную обработку отрицательных чисел.
- Этот код работает для всех списков (включая пустые), но если список пустой, функция вернёт ошибку `TypeError`. Для полной надёжности можно добавить проверку на пустой список:

```
```python
def find_max(lst):
    if not lst:
        return None # или другое значение по умолчанию
    max_val = float('-inf')
    for x in lst:
        if x > max_val:
            max_val = x
    return max_val
```
```

#### json

**\*\*Параметры:\*\*** `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9, 'repeat_penalty': 1.1}`

```
```
{
  "buyer": "Петя",
  "items": [
    {
      "name": "яблоко",
      "count": 3,
      "price": 10
    },
    {
      "name": "апельсин",
      "count": 2,
      "price": 15
    }
  ],
  "total": 60
}
```
```

#### story

```
Параметры: `{'temperature': 0.1, 'num_ctx': 4096, 'top_p': 0.9, 'repeat_penalty': 1.1}`
```

```
...
```

**\*\*Ночь в тумане\*\***

Московский дождь стучал в окно, как будто пытался вырваться из ладони Саши. На экране мигала строка: \*«Почему небо синее?»\*. Он уже третий раз проверял LLM — этот «умный брат», который вчера умудрился объяснить, почему кофе горячий, а не холодный.

— Странно, — мрачно пробормотал он, вводя запрос. Модель ответила: \*«Синий цвет возникает из-за рассеивания света на молекулах кислорода и азота...»\*. Саша усмехнулся. Как будто он спрашивал про солнце в бутылке, а модель отвечала как физик-пожарный.

Он закрыл окно, но через час снова открыл. Утро уже светило, а LLM умело шутить: \*«Дождь — это туман, который не хочет сушиться»\*. Саша поднял брови. Видимо, баг был не в коде, а в том, что он слишком долго думал, чтобы понять, почему небо синее.

\*«Ладно», — сказал он себе. — Утро начинается. И LLM уже знает, как улыбаться в тумане»\*.

(198 слов)

```
...
```

```

```

```
Режим: balanced | Параметры: {'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}
```

```
explain
```

```
Параметры: `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`
```

```
...
```

```
1) Интуитивное объяснение для новичка (4 предложения)
```

Бинарный поиск — это метод для быстрого поиска элемента в **\*\*отсортированном\*\*** списке. Он работает так: делим список на две части и проверяем средний элемент. Если искомый элемент меньше среднего, ищем в левой половине, иначе — в правой. Повторяем процесс, пока не найдем элемент или список не станет пустым. Это позволяет находить элемент за логарифмическое время.

```

```

```
2) Формальное описание шагов
```

1. Проверяем, существует ли массив и не пустой ли он.
2. Находим середину отсортированного массива (индекс `mid = (left + right) // 2``).
3. Сравниваем элемент в `mid`` с искомым значением:
  - Если равны — возвращаем `mid``.
  - Если искомое значение меньше — уменьшаем правую границу до `mid - 1``.
  - Если искомое значение больше — увеличиваем левую границу до `mid + 1``.
4. Повторяем шаги 2–3, пока `left`` не станет больше `right``.
5. Если элемент не найден — возвращаем `-1``.

---

### 3) Код на Python

```
```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        if target < arr[mid]:
            right = mid - 1
        else:
            left = mid + 1
    return -1
```
```

#### bugfix

**Параметры:** `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`

---

1) Код ищет максимальное значение в списке, но при этом не учитывает отрицательные числа, так как инициализация `max_val = 0` приводит к неверному результату для списков с отрицательными числами (например, для списка `[-1, -2]` функция вернёт `0`, а правильный ответ — `-1`).

2) Ошибка в инициализации переменной `max_val`, которая устанавливается равной `0`. Это неверно, так как для списков с отрицательными числами максимальное значение будет меньше нуля.

3) Исправленный код:

```
```python
def find_max(lst):
    max_val = lst[0] # Инициализация с первым элементом списка
    for x in lst:
        if x > max_val:
            max_val = x
    return max_val
```
```

**Пояснение:**

Использование `lst[0]` как начального значения `max_val` гарантирует корректную работу для всех списков (включая отрицательные числа). Предполагается, что список не пуст (в противном случае код будет выбрасывать ошибку `IndexError`). Если требуется обработка пустого списка, нужно добавить проверку на пустой список, но в данном задании это не требуется.

---

#### json

**Параметры:** `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`

---

```
{
 "buyer": "Петя",
 "items": [
```

```

 {
 "name": "яблоко",
 "count": 3,
 "price": 10
 },
 {
 "name": "апельсин",
 "count": 2,
 "price": 15
 }
],
 "total": 60
}
...

```

#### story

**Параметры:** `{'temperature': 0.4, 'num_ctx': 8192, 'top_p': 0.9}`

...

**Ночь в логике**

Ночь в кабинете, где дождь стучал в окно, как будто пытаюсь пронзить стены. Максим, программист с глазами, которые уже не различали между кодом и сном, сидел у монитора. Его руки дрожали, кофе на столе был холодным, а в углу лежал листок с надписью: \*«LLM не дебагится, он дебагит»\*.

Он пытался найти баг в модели, которая уже три ночи не спала. «Ты слишком устал, – шептал LLM, его голос звучал в ушах, как будто из тумана. – Давай просто уснёшь». Максим смеялся, но в глубине души понимал: модель не ошибалась. Она просто знала, что он уже не мог различить, где код, а где его собственный мозг.

Когда он уснул, в его телефон пришло сообщение: \*«Спокойной ночи, Максим. Я уже дебагил тебя»\*.

В итоге он не знал, что было багом – в коде или в том, что он сам уже не мог различить, что дебагит.

(198 слов)

...

---

### Режим: creative | Параметры: `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`

#### explain

**Параметры:** `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`

...

1) Бинарный поиск – это алгоритм для быстрого поиска элемента в отсортированном списке. Он сравнивает искомое значение с серединным элементом текущего диапазона. Если искомое меньше середины – ищем в левой половине, иначе – в правой. На каждом шаге алгоритм исключает половину элементов, что делает поиск логарифмически быстрым. Это позволяет найти элемент за  $O(\log n)$  времени, что значительно эффективнее линейного поиска.

2) Формальное описание шагов:

1. Установите левый индекс в начало списка (0), правый индекс в конец списка (n-1).
2. Повторяйте:
  - a. Найдите середину диапазона:  $mid = (left + right) // 2$ .
  - b. Если `arr[mid]` равно искомому значению – верните `mid`.
  - c. Если искомое меньше `arr[mid]` – установите правый индекс в `mid - 1`.
  - d. Если искомое больше `arr[mid]` – установите левый индекс в `mid + 1`.
3. Если цикл завершился без нахождения элемента, верните -1.

3) Код на Python:

```
```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif target < arr[mid]:
            right = mid - 1
        else:
            left = mid + 1
    return -1
```
```

#### bugfix

```
```python
**Параметры:** `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`
```
```

1) **Что делает код?**

Код ищет максимальное значение в списке `lst`, но имеет ошибку в инициализации переменной `max_val`. Если список содержит только отрицательные числа, функция вернет `0` вместо правильного максимального значения (например, для `[-1, -2, -3]` функция вернет `0`, а правильный ответ – `-1`).

2) **Найди ошибку**

Ошибка в инициализации `max_val = 0`. Это приводит к некорректному результату, если в списке есть отрицательные числа. Начальная величина `0` не учитывает возможные отрицательные значения, поэтому функция не корректно определяет максимум.

3) **Исправь код**

Исправленный код инициализирует `max_val` значением первого элемента списка (`lst[0]`), чтобы правильно обрабатывать отрицательные числа. Также добавлена проверка на пустой список для безопасного выполнения.

```
```python
def find_max(lst):
    if not lst:
        raise ValueError("Список не должен быть пустым")
    max_val = lst[0]
    for x in lst[1:]:
        if x > max_val:
            max_val = x
```
```

```

 return max_val
...

Пояснение к исправлению:
- Проверка на пустой список (`if not lst`) предотвращает ошибку `IndexError`
при работе с пустым списком.
- Начинаем с первого элемента списка (`lst[0]`), чтобы корректно обрабатывать
отрицательные числа и другие случаи.
- Цикл проходит по оставшимся элементам списка (`lst[1:]`), сравнивая каждый
элемент с текущим `max_val`.
...

json
Параметры: `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`
...
{
 "buyer": "Петя",
 "items": [
 {
 "name": "яблоко",
 "count": 3,
 "price": 10
 },
 {
 "name": "апельсин",
 "count": 2,
 "price": 15
 }
],
 "total": 60
}
...

story
Параметры: `{'temperature': 0.9, 'num_ctx': 4096, 'top_p': 0.95}`
...

Ночь в квартале, где туман не проходит сквозь окна. Саша сидит за
компьютером, экран мерцает, как будто пытается увидеть его в темноте. Ветер
шепчет уличные фонари, как будто дебажит его сон.

Он вводит в LLM: «Что такое нормальная работа?» Модель отвечает: «Нормальная
работа – это когда модель не может понять, что такое нормальная работа. Её
задача – уйти в бесконечность». Саша улыбается, но в душе знает, что это не
шутка. Он пытается найти ошибку в коде, но LLM начинает отвечать на русском:
«Вы спите. Это нормально». Саша смеётся – но это не смех, а размышление о
том, что он и модель в одиночку в тупой тишине.

В 3 утра он закрывает экран. В комнате тишина. LLM уснул. Саша тоже. И они
оба понимают: «Нормальная работа – это когда ты не можешь понять, что не
работает». Он думает: «Дебагу ли я модель или модель дебагу меня?». Ветер
умолкает. В тишине только тихий звук от костра, который уже не горит.

(199 слов)
...

```

---

## ## Сводная таблица

| Модель    | Режим    | Промпт  | Токены | Длина ответа |
|-----------|----------|---------|--------|--------------|
| gemma3:1b | strict   | explain | 809    | 2726         |
| gemma3:1b | strict   | bugfix  | 435    | 1606         |
| gemma3:1b | strict   | json    | 82     | 165          |
| gemma3:1b | strict   | story   | 342    | 1192         |
| gemma3:1b | balanced | explain | 688    | 2229         |
| gemma3:1b | balanced | bugfix  | 436    | 1594         |
| gemma3:1b | balanced | json    | 100    | 213          |
| gemma3:1b | balanced | story   | 401    | 998          |
| gemma3:1b | creative | explain | 783    | 2545         |
| gemma3:1b | creative | bugfix  | 470    | 1711         |
| gemma3:1b | creative | json    | 100    | 213          |
| gemma3:1b | creative | story   | 354    | 934          |
| qwen3:4b  | strict   | explain | 914    | 1200         |
| qwen3:4b  | strict   | bugfix  | 2632   | 1385         |
| qwen3:4b  | strict   | json    | 1333   | 201          |
| qwen3:4b  | strict   | story   | 1470   | 879          |
| qwen3:4b  | balanced | explain | 1272   | 1295         |
| qwen3:4b  | balanced | bugfix  | 2221   | 1036         |
| qwen3:4b  | balanced | json    | 1370   | 201          |
| qwen3:4b  | balanced | story   | 1218   | 837          |
| qwen3:4b  | creative | explain | 2518   | 1206         |
| qwen3:4b  | creative | bugfix  | 1817   | 1385         |
| qwen3:4b  | creative | json    | 1960   | 201          |
| qwen3:4b  | creative | story   | 3084   | 932          |